TechCorp Prompt Engineering Standards

Guidelines for LLM Prompt Design & Management

Version: v2.0	Document: TC-PE-003	TechCorp Internal

1. Prompt Writing Principles

All prompts deployed to production must follow TechCorp's Clarity-Context-Constraint framework (C3). Every prompt should be clear about the task, provide sufficient context for grounding, and constrain the model to the expected output format.

- Be explicit: never assume the model will infer intent from ambiguous wording
- Use positive framing: state what TO do, not only what NOT to do
- Separate instructions from data using clear delimiters (XML tags preferred)
- Include output format specification in every prompt
- Version all prompts — semantic versioning required (e.g., v1.2.3)
- Store prompts in the Prompt Registry, never hard-coded in application logic

2. System Prompt Rules

Max Length	4,000 tokens — exceeding this requires Architecture review
Structure	Role → Task → Constraints → Output Format → Examples
Persona	Define a named, scoped persona (e.g., 'You are TechCorp Support Agent')
Tone	Specify explicitly: professional, empathetic, concise
Forbidden	Never include API keys, PII, or customer data in system prompts
Review Gate	All system prompt changes require peer review + AI Safety sign-off
Fallback	Must include instructions for out-of-scope queries

3. User Prompt Rules

- Sanitize all user input before injection: strip control characters, truncate at 2,000 tokens
- Never concatenate raw user input directly into a system prompt
- Validate that user-provided context does not contain prompt injection patterns

- Apply PII detection before logging any user prompt
- Rate-limit user prompts: 60 requests/minute per authenticated user
- Log prompt hash (SHA-256) for audit trail — never log raw user prompt in plain text

---

4. Context Injection

For RAG-powered prompts, retrieved context must be injected using the canonical TechCorp context template. Context window budget allocation:

System Prompt	~800 tokens (reserved)
Retrieved Context	~2,500 tokens (dynamic, trimmed by relevance score)
Conversation History	~1,000 tokens (last 5 turns, summarized if longer)
User Query	~200 tokens (hard limit, truncated with warning)
Output Buffer	1,500 tokens (reserved for model response)
Total Budget	~6,000 tokens (claude-sonnet-4 / gpt-4o 128k context)

---

5. RAG Prompt Template

Canonical RAG system prompt structure:

```
You are {persona_name}, a {role} at TechCorp. Answer questions using ONLY the context below. If the answer is not in the context, say: 'I don't have that information.' Do not fabricate facts. Be concise and professional. {retrieved_chunks} {user_query}
```

---

6. Hallucination Prevention

- Instruct the model to cite source document IDs for every factual claim
- Use temperature ≤ 0.3 for fact-retrieval tasks; 0.7 for creative/generative tasks
- Enable 'grounded generation' mode — model must quote relevant passage before answering
- Post-process responses with a fact-check pass using a separate critic model
- Flag responses where model confidence (logprob threshold) falls below 0.6
- Never use streaming for high-stakes responses — validate full output before display

---

7. Output Formatting

Structured Data	Request JSON with explicit schema; validate with Pydantic
Long-form Text	Request Markdown with H2/H3 headings; render client-side
Tables	Request pipe-delimited Markdown tables
Code Snippets	Request fenced code blocks with language specifier

Lists	Use numbered lists for ordered steps; bullets for unordered items
Confidence	Request explicit confidence level: HIGH / MEDIUM / LOW

8. Example Prompts

Example: Customer Support Query Classification

```
Classify the following customer message into exactly one category. Categories:
BILLING | TECHNICAL | ACCOUNT | GENERAL Respond with ONLY the category label, nothing
else. Message: {user_message}
```

Example: Document Summarization

```
Summarize the following document in 3 bullet points. Each bullet must be under 20
words. Use professional, neutral tone. {document_text}
```